

Python Application Programming

Anitya Gupta

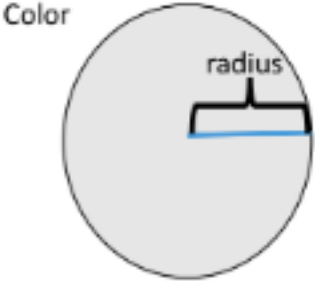
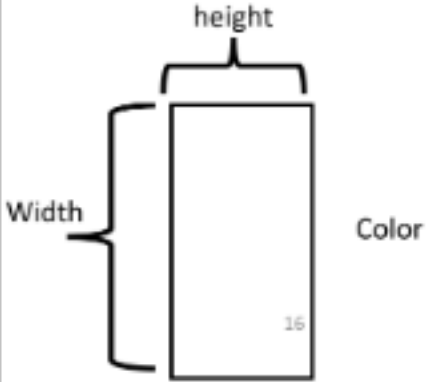
Classes

- A class is a blueprint or template for creating objects. It defines the structure and behavior that its objects will have.
- In Python, you can create classes using the class keyword.

Creating Classes

- When you create a class, you specify the attributes(data) and methods (functions) that objects of that class will have.
- Attributes are defined as variables within the class, and methods are defined as functions.

Classes

Class Circle	Class Rectangle
Attributes: radius, Color	Attributes: Color, height and Width
	

The class Circle has the attribute radius and color, while the Rectangle class has the attribute height and width.

Objects

- An object is a fundamental unit in Python that represents a real-world entity or concept. Objects can be tangible (like a car) or abstract (like a student's grade). Every object has two main characteristics:
 - State : The attributes or data that describe the object. For your "Car" object, this might include attributes like "color", "speed", and "fuel level".
 - Behavior: The actions or methods that the object can perform. In Python, methods are functions that belong to objects and can change the object's state or perform specific operations.

Instances

- Once you've defined a class, you can create individual objects (instances) based on that class.
- Each object is independent and has its own set of attributes and methods.
- To create an object, you use the class name followed by parentheses, so: `"my_car = Car()"`

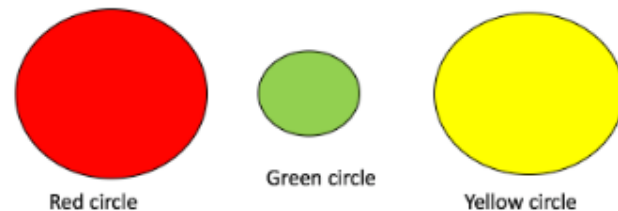


Figure 2: Three instances of the class Circle, or three objects of type Circle.

Methods

- Methods give you a way to change or interact with the object; they are functions that interact with objects.
- For example, let's say we would like to increase the radius of a circle by a specified amount. We can create a method called **add_radius(r)** that increases the radius by **r**.

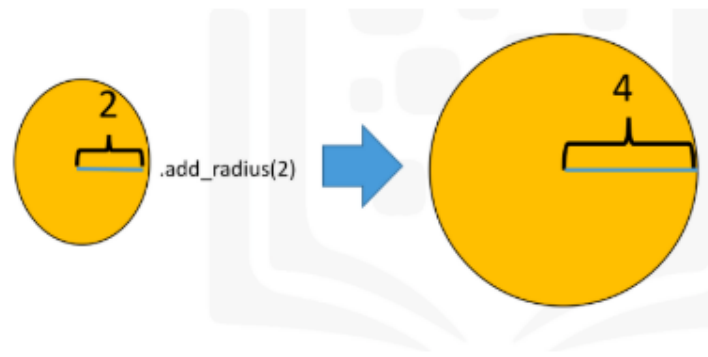


Figure 3: Applying the method "add_radius" to the object orange circle object.

Constructor method (def init(self, attribute1, attribute2, ...):)

- The `__init__` method is a special method known as the constructor.
- It initializes the instance attributes (also called instance variables) when an object is created.
- The `self` parameter is the first parameter of the constructor, referring to the instance being created.
- `attribute1`, `attribute2`, and so on are parameters passed to the constructor when creating an object.
- Inside the constructor, `self.attribute1`, `self.attribute2`, and so on are used to assign values to instance attributes.

Creating a Class

```
class Circle(object):  
    def __init__(self, radius, color):  
        self.radius = radius;  
        self.color = color;  
    def add_radius(self, r):  
        self.radius = self.radius + r  
        return self.radius
```

} Define your class

} Data attributes used to initialize object

} Method used to add r to radius

Figure 5: Labeled syntax of the object circle.

Instance attributes (`self.attribute1 = attribute1`)

- Instance attributes are variables that store data specific to each class instance.
- They are initialized within the `__init__` method using the `self` keyword followed by the attribute name.
- These attributes hold unique data for each object created from the class.

Instance methods (def method1(self, parameter1, parameter2, ...):)

- Instance methods are functions defined within the class.
- They operate on the instance's data (instance attributes) and can perform actions specific to instances.
- The **self** parameter is required in instance methods, allowing them to access instance attributes and call other methods within the class.

Calling Methods on Objects

- The methods `method1` and `method2` are defined in the `ClassName` class, and you're calling them on `object1` and `object2` respectively.
- You pass values `param1_value` and `param2_value` as arguments to these methods. These arguments are used within the method's logic.

Method 1: Using dot notation

- This is the most straightforward way to call an object's method. In this, use the dot notation (`object.method()`) to invoke the method on the object directly.
- For example, `result1 = object1.method1(param1_value, param2_value, ...)` calls `method1` on `object1`.

```
1  # Calling methods on objects
2  # Method 1: Using dot notation
3  result1 = object1.method1(param1_value, param2_value, ...)
4  result2 = object2.method2(param1_value, param2_value, ...)
```



Method 2: Assigning object methods to variables

- Here's an alternative way to call an object's method by assigning the method reference to a variable.
- `method_reference = object1.method1` assigns the method `method1` of `object1` to the variable `method_reference`.
- Later, call the method using the variable like this: `result3 = method_reference(param1_value, param2_value, ...)`

```
1  # Method 2: Assigning object methods to variables
2  method_reference = object1.method1 # Assign the method to a variable
3  result3 = method_reference(param1_value, param2_value, ...)
```

